Teoria dei sistemi e Architettura

Raccolta Completa: 4 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

Architettura e Funzionamento del Computer

> **Obiettivo**
>
> Questo appunto approfondisce l'architettura interna del calcolatore digitale, dal modello teorico di Von Neumann all'esecuzione delle istruzioni nella CPU (pipeline, RISC vs CISC), la gerarchia delle memorie, la sicurezza del boot hardware/software e gli standard moderni di connessione e I/O.

---

1. Il Modello di Von Neumann

La quasi totalità dei calcolatori moderni si basa sul modello teorizzato dal matematico **John von Neumann** nel 1945. La sua intuizione rivoluzionaria fu il concetto di **programma memorizzato**: dati e istruzioni di programma risiedono nello stesso spazio di memoria fisica e sono trattati entrambi in formato binario.

```
flowchart TD
    subgraph CPU ["CPU (Central Processing Unit)"]
        CU["Control Unit (CU)<br/>• Program Counter (PC)<br/>• Instruction Register
        ALU["Arithmetic Logic Unit (ALU)<br/>• Calcoli aritmetici e logici<br/>• Acc
        REG["Registri Interni (MAR, MDR, Status)"]
        CU <--> ALU
        CU <--> REG
        ALU <--> REG
    end

    RAM[("Memoria Centrale (RAM)<br/>Celle indirizzabili a Byte<br/>Dati e Istruzior
    IO["Sistema di Input/Output (I/O)<br/>Interfacce e Periferiche"]

    BUS_D["Bus Dati (Bidirezionale)"]
    BUS_A["Bus Indirizzi (Unidirezionale: CPU → RAM/IO)"]
    BUS_C["Bus Controllo (Segnali R/W, Clock, Interrupt)"]

    CPU <==> BUS_D
    CPU ==> BUS_A
    CPU <==> BUS_C

    RAM <==> BUS_D
    BUS_A ==> RAM
    BUS_C <==> RAM

    IO <==> BUS_D
    BUS_A ==> IO
    BUS_C <==> IO
```

Componenti Principali:

1. Unità Centrale di Elaborazione (CPU)

Il “motore” del computer, incaricato di prelevare ed eseguire le istruzioni:

- **Control Unit (CU - Unità di Controllo):** Coordina tutte le attività del calcolatore, preleva l’istruzione dalla RAM, la decodifica e invia segnali di comando temporizzati a tutti i sottosistemi.

- **Arithmetic Logic Unit (ALU - Unità Aritmetico-Logica):** Esegue le operazioni matematiche (somma, sottrazione, moltiplicazione) e logiche (AND, OR, NOT, confronto).
- **Registri Interni:** Piccolissime celle di memoria ad altissima velocità situate direttamente nel silicio del processore:
  - **PC (Program Counter):** Contiene l'indirizzo della *prossima* istruzione da eseguire.
  - **IR (Instruction Register):** Contiene l'istruzione binaria correntemente in fase di decodifica/esecuzione.
  - **MAR (Memory Address Register):** Contiene l'indirizzo di memoria a cui si vuole accedere in lettura o scrittura.
  - **MDR (Memory Data Register):** Contiene il dato letto dalla memoria o pronto per esservi scritto.
  - **Status Register (Flags):** Memorizza lo stato del processore dopo un'operazione (segno negativo, zero, overflow, riporto).
- **Cache (L1, L2, L3):** Memorie statiche ultra-veloci integrate nel chip per minimizzare i tempi di attesa dell'accesso alla RAM.

2. Memoria Centrale (RAM - Random Access Memory)

- Memoria volatile ad accesso casuale (il tempo di accesso è costante e indipendente dall'indirizzo).
- Organizzata in una sequenza ordinata di **celle**, ciascuna dotata di un **indirizzo univoco**.
- Contiene i dati e i programmi attualmente caricati ed eseguiti dalla CPU.

3. Sistema di Input/Output (I/O)

Permette l'interazione tra il computer e il mondo esterno:

- **Input:** Riceve dati dall'esterno e li converte in formato binario (es. tastiera, mouse, webcam).
- **Output:** Traduce i dati elaborati in un formato fruibile dall'uomo o da altri sistemi (es. monitor, altoparlanti, stampante).

4. I Bus di Sistema

Linee fisiche di collegamento in rame o piste stampate su scheda madre:

- **Bus Indirizzi (Address Bus):** *Unidirezionale*. La CPU invia l'indirizzo della cella di memoria o del dispositivo I/O su cui operare. L'ampiezza in bit determina lo spazio di memoria indirizzabile (2^N byte, con N bit di bus indirizzi; es. 32 bit = 4 GB, 64 bit = 16 Exabyte).

- **Bus Dati (Data Bus):** *Bidirezionale*. Trasporta i byte effettivi delle istruzioni e dei dati tra CPU, RAM e I/O.
- **Bus di Controllo (Control Bus):** *Misto/Bidirezionale*. Trasporta segnali di sincronizzazione e comando (segnale di Clock, Read/Write, Interrupt Request `IRQ`, Bus Grant, Reset).

---

2. Il Ciclo Macchina e le Architetture di Istruzione

Il Ciclo di Esecuzione (Fetch-Decode-Execute)

Ogni istruzione di un programma attraversa un ciclo rigorosamente sincronizzato dal clock della CPU:

```
flowchart TD
    F["1. FETCH (Prelievo)<br/>• PC invia indirizzo al MAR<br/>• Lettura memoria → M
    D["2. DECODE (Decodifica)<br/>• La CU interpreta il codice operativo (OpCode)<br
    E["3. EXECUTE (Esecuzione)<br/>• L'ALU compie il calcolo o<br/>• Viene attivata
    W["4. WRITE-BACK (Scrittura Risultato)<br/>• Il risultato viene salvato nei Regi

    F --> D --> E --> W --> F
```

Pipelining delle Istruzioni

Nei processori tradizionali non-pipelined, ogni istruzione deve completare tutte le 4 fasi prima che la successiva possa iniziare. Con il **Pipelining**, le fasi vengono sovrapposte come in una catena di montaggio industriale:

Ciclo di Clock	T1	T2	T3	T4	T5	T6
Istruzione 1	Fetch	Decode	Execute	WriteBack	-	-
Istruzione 2	-	Fetch	Decode	Execute	WriteBack	-
Istruzione 3	-	-	Fetch	Decode	Execute	WriteBack

> **Risultato del Pipelining**
>
> A regime, il processore completa **un'istruzione per ogni ciclo di clock**, moltiplicando le prestazioni complessive (*throughput*).

Confronto Architetturale: RISC vs CISC

```
flowchart LR
    subgraph CISC [Architettura CISC - Complex]
        C1["Istruzioni ricche e complesse"] --> C2["Numero variabile di cicli clock"
        C2 --> C3["Accesso diretto alla RAM nelle operazioni"]
        C3 --> C4["Esempio: Famiglia Intel x86 / AMD64"]
    end

    subgraph RISC [Architettura RISC - Reduced]
        R1["Istruzioni semplici e uniformi (32 bit fissi)"] --> R2["1 istruzione per
        R2 --> R3["Architettura Load / Store<br/>(operazioni solo su registri interni
        R3 --> R4["Esempio: ARM, Apple Silicon (M-series), RISC-V"]
    end
```

Parametro	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Set di istruzioni	Molto ampio (centinaia di istruzioni complesse)	Ridotto ed essenziale
Durata istruzione	Variabile (da 1 a decine di cicli di clock)	Fissa (1 ciclo per istruzione in pipeline)
Formato istruzioni	Lunghezza variabile (da 1 a 15 byte)	Lunghezza fissa (es. 32 bit o 64 bit)
Accesso alla Memoria	Molte istruzioni operano direttamente su RAM	Solo istruzioni dedicate `LOAD` e `STORE`
Efficienza Energetica	Più energivoro (richiede decodificatori complessi)	Altamente efficiente (ideale per smartphone ed embedded)

---

3. Gerarchia delle Memorie e Sistemi RAID

Nessuna tecnologia di memoria possiede contemporaneamente massima velocità, massima capienza e costo minimo. L'architettura adotta quindi una **struttura gerarchica a piramide**:

```
flowchart TD
    R["1. Registri CPU (Capacità: < 1 KB, Latenza: < 1 ns)"]
    C["2. Cache L1, L2, L3 (SRAM, Capacità: pochi MB, Latenza: pochi ns)"]
    M["3. Memoria Principale (DRAM, Capacità: 8 - 128 GB, Latenza: ~50-80 ns)"]
    S["4. Memoria Secondaria SSD/NVMe (NAND Flash, Centinaia GB - TB, Latenza: ~10-1
    H["5. Memoria Meccanica HDD (Dischi Magnetici, TeraByte, Latenza: ~5-15 ms)"]
    N["6. Memoria Terziaria / Offline (Nastri magnetici, Cloud Storage a freddo)"]

    R --- C --- M --- S --- H --- N
```

> **Regola della Gerarchia**

> Salendo verso il vertice: **aumenta la velocità** e il **costo per bit**, ma **diminuisce la capacità**. Scendendo verso la base: aumenta la capacità e diminuisce il costo, ma aumentano i tempi di latenza.

Tecnologia RAID (Redundant Array of Independent Disks)

Combina più dischi fisici indipendenti per incrementare la velocità, la tolleranza ai guasti o entrambe:

Livello RAID	Nome / Principio	Tolleranza Guasti	Velocità	Spazio Utile	Note di Utilizzo
RAID 0	Striping (Dati suddivisi a blocchi alternati tra dischi)	❌ **Nessuna** (se 1 disco cede, si perde tutto)	⚡ Doppia / Moltiplicata	100% (N × disco)	Solo per elaborazioni video o gaming senza dati critici.
RAID 1	Mirroring (Copia speculare esatta su due o più dischi)	✅ Tolleranza al guasto di 1 disco	Velocità di lettura migliorata, scrittura normale	50% (dimezzato)	Massima sicurezza per sistemi operativi e database critici.
RAID 5	Striping con Parità Distribuita (minimo 3 dischi)	✅ Tolleranza al guasto di **1 disco**	Elevata lettura, scrittura con calcolo parità	(N – 1) × disco	Ottimo bilanciamento costo/prestazioni nei server aziendali.
RAID 10 (1+0)	Mirroring + Striping (minimo 4 dischi)	✅ Tolleranza al guasto di 1 disco per coppia speculare	⚡ Altissima velocità sia in lettura che in scrittura	50%	Ideale per server di produzione mission-critical.

---

4. Il Processo di Avvio del Computer (Boot Sequence)

All'accensione, la memoria RAM è completamente vuota e la CPU non ha alcun programma da eseguire. Per risolvere questo "stallo iniziale", il processore è cablato per puntare a un indirizzo fisso nella memoria non-volatile:

```
flowchart TD
    Accensione["1. Pressione Tasto Accensione / Alimentatore eroga tensioni stabili
    ResetVec --> POST["3. POST (Power-On Self-Test): Controllo diagnostico CPU, RAM,
    POST --> Firmware["4. BIOS / UEFI inizializza le periferiche essenziali"]
    Firmware --> SecBoot{"5. Secure Boot abilitato?"}
    SecBoot -- Sì --> CheckSig["Verifica firma digitale autorizzata del Bootloader"]
    SecBoot -- No --> LoadBoot["Lettura del Bootloader dal supporto di avvio"]
    CheckSig --> LoadBoot
    LoadBoot --> Kernel["6. Il Bootloader carica il Kernel del Sistema Operativo in
    Kernel --> Init["7. Inizializzazione Driver, Servizi e Shell Utente (Login)"]
```

Ruolo di BIOS e UEFI

- **BIOS Legacy (Basic Input/Output System):** Sistema storico a 16 bit, supporta solo partizionamento MBR (limite massimo dischi a 2 TB e 4 partizioni primarie).
- **UEFI (Unified Extensible Firmware Interface):** Standard moderno a 32/64 bit con interfaccia grafica, supporto al partizionamento GPT (dischi oltre 2 TB e partizioni virtualmente illimitate) e supporto a **Secure Boot**.

---

5. Fasi di Avvio del Sistema Operativo Windows

Nei moderni sistemi basati su UEFI e Windows 10/11, la sequenza di avvio si articola in 4 fasi:

```
sequenceDiagram
    autonumber
    participant HW as Firmware UEFI
    participant BM as Windows Boot Manager (bootmgr.efi)
    participant BL as Windows OS Loader (winload.efi)
    participant KRNL as Kernel (ntoskrnl.exe & HAL)
    participant USR as Session Manager & Winlogon

    HW->>BM: Avvia bootmgr.efi dalla partizione EFI (ESP)
    BM->>BL: Legge il BCD (Boot Configuration Data) e lancia winload.efi
    BL->>KRNL: Carica in RAM ntoskrnl.exe, hal.dll e i driver di boot
    KRNL->>USR: Inizializza i driver, avvia smss.exe, csrss.exe e winlogon.exe
    USR-->>HW: Presenta la schermata di Logon Utente
```

Tecnologie di Sicurezza Hardware:

- **TPM (Trusted Platform Module 2.0):** Microchip crittografico dedicato integrato sulla scheda madre. Custodisce in hardware le chiavi crittografiche per **BitLocker** (crittografia disco completo) e garantisce l'integrità del sistema certificando che il firmware e il bootloader non siano stati manomessi.
- **Secure Boot:** Blocca l'esecuzione di bootkit o driver malevoli prima del caricamento del sistema operativo, accettando solo codice firmato digitalmente da un'autorità fidata (Microsoft o costruttore OEM).

---

6. Periferiche e Bus di Connessione

Classificazione delle Periferiche

- **Input:** Tastiera, mouse, scanner, tavoletta grafica, microfono.
- **Output:** Monitor, videoproiettore, stampante, casse audio.
- **Input/Output (I/O):** Schede di rete Ethernet/Wi-Fi, memorie di massa (dischi SSD, pendrive USB), touchscreen.

Standard di Connessione Principali:

Bus / Interfaccia	Tipologia	Velocità Tipica	Utilizzo Principale
PCI Express (PCIe)	Seriale interna punto-punto ad altissime prestazioni (da x1 a x16 corsie)	Gen 4: ~2 GB/s per corsia (x16 = 32 GB/s) Gen 5: ~4 GB/s per corsia (x16 = 64 GB/s)	Schede video dedicate (GPU), SSD NVMe M.2 ultra-veloci, schede di rete 10/100 GbE.
SATA III	Seriale interna per memorie di massa	6 Gbps (circa 550-600 MB/s reali)	Hard disk meccanici (HDD) e SSD tradizionali da 2.5 pollici.
M.2 NVMe	Form-factor compatto su bus PCIe	Fino a 7.000 - 14.000 MB/s	SSD moderni ad altissima velocità su schede madri e laptop.
USB (Universal Serial Bus)	Seriale esterna universale plug-and-play	• USB 2.0: 480 Mbps • USB 3.2 Gen 2: 10 Gbps • USB4 / Thunderbolt 4: 40 - 80 Gbps	Mouse, tastiere, dischi esterni, audio, video e ricarica Power Delivery (con connettore Type-C).
HDMI / DisplayPort	Bus digitali video/audio ad alta definizione	Da 18 Gbps (HDMI 2.0) a 80 Gbps (DisplayPort 2.1)	Monitor per computer, TV 4K/8K, frequenze di refresh elevate (144Hz+).
VGA / DVI (Legacy)	Video analogico (VGA) o digitale prima generazione (DVI)	Limitata a risoluzioni Full HD o inferiori	Standard obsoleti gradualmente dismessi.

Teoria dei Sistemi: Fondamenti e Modellazione

> **Obiettivo della trattazione**
>
> Comprendere cos'è un sistema, come viene schematizzato attraverso il modello a scatola nera (*black box*), come si descrivono gli stati interni e in che modo le funzioni di transizione e trasformazione ne determinano l'evoluzione temporale.

---

1. Cos'è un Sistema?

Un **sistema** è un insieme di elementi o componenti interconnessi tra loro che interagiscono in modo coordinato secondo leggi ben definite per raggiungere un **obiettivo comune** (uno scopo o una funzione specifica).

```
flowchart LR
    A["Ambiente Esterno"] -->|Ingressi (Stimoli)| S["SISTEMA<br/>(Insieme di compone
    S -->|Uscite (Risposte)| A
```

Concetti cardine:

- **Elementi (Componenti):** Le singole parti costitutive del sistema.
- **Interazioni (Relazioni):** I legami funzionali, logici o fisici tra le parti.
- **Confine del sistema:** La linea di separazione (reale o concettuale) che delimita ciò che appartiene al sistema rispetto all'ambiente esterno.
- **Ambiente esterno:** Tutto ciò che si trova al di fuori del confine e con cui il sistema scambia informazioni, energia o materia.

---

2. Sistemi, Sottosistemi ed Elementi

La definizione di ciò che è “sistema” dipende sempre dalla scala di osservazione (*punto di vista dello studio*):

```mermaid
graph TD
    Auto["Sistema Principale: Automobile<br/>(Scopo: Mobilità)"]
    Auto --> Motore["Sottosistema: Motore<br/>(Scopo: Generare potenza)"]
    Auto --> Freno["Sottosistema: Impianto Frenante<br/>(Scopo: Decelerazione)"]
    Auto --> Scarico["Sottosistema: Scarico<br/>(Scopo: Evacuazione fumi)"]
    Motore --> Candela["Elemento: Candela"]
    Motore --> Pistone["Elemento: Pistone"]
```

- **Sottosistema:** Se analizziamo una parte complessa del sistema (es. il *motore* dell’automobile), essa è a sua volta un sistema autonomo con propri componenti e scopi specifici. Pur agendo singolarmente, concorre al raggiungimento dell’obiettivo del sistema principale.
- **Elemento semplice:** Se una parte non viene ulteriormente scomposta e viene considerata come indivisibile nel contesto della nostra analisi (es. un singolo bullone o la candela), essa prende il nome di *elemento*.

---

3. Il Modello a Scatola Nera (*Black Box*)

Nello studio ingegneristico dei sistemi non è sempre necessario o conveniente conoscere ogni singolo dettaglio costruttivo interno. Spesso si adotta l’approccio a **Scatola Nera (Black Box)**: si osserva il sistema solo dal punto di vista delle relazioni di causa-effetto tra ciò che entra e ciò che esce.

```mermaid
flowchart LR
    subgraph BLACK_BOX [Modello a Scatola Nera]
        direction LR
        S[("Stato Interno: S(t)")]
    end
    I["Ingressi: I(t)"] --> BLACK_BOX
    BLACK_BOX --> U["Uscite: U(t)"]
```

Un sistema è formalmente caratterizzato da tre vettori di variabili:

1. Ingressi (I)

Le grandezze provenienti dall'ambiente esterno che sollecitano il sistema: $I = \{i1, i2, \dots, ik\}$ Ogni ingresso ij può assumere valori appartenenti a un insieme o alfabeto di ingresso Vij.

2. Uscite (U)

Le risposte che il sistema invia verso l'ambiente esterno: $U = \{u1, u2, \dots, um\}$ Ogni uscita uj assume valori nell'insieme dei valori di uscita Vuj.

3. Stati Interni (S)

L'insieme delle informazioni minime necessarie all'istante t che, insieme alla conoscenza degli ingressi futuri, determinano univocamente il comportamento futuro del sistema. $S = \{s1, s2, \dots, sn\}$

> **Intuizione visiva dello "Stato"**
>
> Lo stato è come una **fotografia istantanea** del sistema scattata all'istante di tempo t. Rappresenta la memoria storica del sistema: conserva la traccia di tutti gli ingressi passati che influenzano il presente.

---

4. Le Funzioni Matematiche del Sistema

Per descrivere completamente il comportamento e l'evoluzione temporale di un sistema a stati discreti, si utilizzano due funzioni fondamentali:

A. La Funzione di Transizione di Stato (f)

Determina quale sarà lo **stato successivo** del sistema al passo temporale successivo ($tk+1$ o $t + 1$), combinando lo **stato attuale** ($S(t)$) e l'**ingresso corrente** ($I(t)$):

$S(t+1) = f(S(t), I(t))$

- **Cosa fa:** Specifica per ogni coppia `(stato corrente, ingresso attuale)` lo spostamento verso il nuovo stato.

B. La Funzione di Trasformazione delle Uscite (g)

Determina il valore dell'**uscita** generata dal sistema:

1. **Modello di Mealy (Sistema Improprio):** L'uscita dipende sia dallo stato attuale sia dall'ingresso attuale: $U(t) = g(S(t), I(t))$
2. **Modello di Moore (Sistema Proprio):** L'uscita dipende **esclusivamente** dallo stato interno attuale: $U(t) = g(S(t))$

---

5. Esempio Pratico: Il Circuito Interruttore-Lampadina

Analizziamo il classico esempio descritto negli appunti: un circuito formato da un interruttore manuale e una lampadina.

```
flowchart LR
    Int["Interruttore (Ingresso i₁)"] --> Circuito[("Circuito Lampadina (Stato S)")]
    Circuito --> Luce["Emissione Luminosa (Uscita u₁)"]
```

Definizione Formale:

- **Ingresso (I):** $I = \{i1\}$, con $Vi1 = \{Aperto, Chiuso\}$
- **Stato (S):** $S = \{s1, s2\}$:
  - s1: Lampadina Spenta (filamento freddo / a riposo)
  - s2: Lampadina Accesa (filamento incandescente / in conduzione)
- **Uscita (U):** $U = \{u1\}$, con $Vu1 = \{Buio, Luce\}$

Tabella di Transizione (Stato Futuro $S(t+1)$)

Indica in quale stato si troverà la lampada in base allo stato attuale e alla posizione dell'interruttore:

Stato Corrente S(t)	Ingresso: Aperto	Ingresso: Chiuso
s1 (Spenta)	s1 (Spenta)	s2 (Accesa)
s2 (Accesa)	s1 (Spenta)	s2 (Accesa)

Tabella di Trasformazione (Uscita U(t))

Indica la luminosità emessa dal sistema:

Stato Corrente S(t)	Ingresso: Aperto	Ingresso: Chiuso
s1 (Spenta)	Buio	Luce (o transitorio verso accesa)
s2 (Accesa)	Buio	Luce

(Se considerata come macchina di Moore pura, l'uscita dipenderebbe direttamente solo dallo stato: s1 → Buio, s2 → Luce).

---

6. Schema di Riepilogo

Componente	Simbolo	Significato	Esempio Pratico
Ingresso	$I(t)$	Stimolo proveniente dall'esterno	Pressione pulsante, moneta inserita
Stato	$S(t)$	Memoria interna all'istante t	Valore di conteggio, marcia inserita
Uscita	$U(t)$	Azione o risultato visibile all'esterno	Accensione LED, erogazione bibita
Transizione (f)	$S(t+1) = f(S, I)$	Regola di cambio di stato	Se ho 1 EUR e inserisco un altro 1 EUR → Stato "2 EUR inseriti"
Trasformazione (g)	$U(t) = g(S, I)$	Regola di produzione dell'uscita	Se ho raggiunto 2 EUR e premo il tasto → Eroga bottiglia

Le Porte Logiche e l'Algebra Booleana

> **Concetto Base**
>
> Le **porte logiche** sono i blocchi elementari costitutivi di qualsiasi circuito elettronico digitale e microprocessore. Ciascuna porta logica riceve uno o più segnali binari in ingresso (0 o 1, fisicamente rappresentati da livelli di tensione) ed elabora **un'unica uscita binaria** secondo le regole dell'algebra di George Boole.

---

1. La Tavola di Verità (*Truth Table*)

La **tavola di verità** è lo strumento matematico fondamentale per descrivere univocamente il comportamento di una porta o di una rete logica. Elenca tutte le possibili combinazioni dei valori di ingresso e la corrispondente uscita prodotta.

Per un circuito con N ingressi binari, la tavola di verità conterrà esattamente $2N$ righe:

- Con 2 ingressi: $2^2 = 4$ combinazioni (`00` , `01` , `10` , `11`).
- Con 3 ingressi: $2^3 = 8$ combinazioni.

---

2. Le Porte Logiche Fondamentali e Derivate

```
flowchart LR
    subgraph FONDAMENTALI [Porte Fondamentali]
        P_NOT["NOT (Inversione)"]
        P_AND["AND (Prodotto Logico)"]
        P_OR["OR (Somma Logica)"]
    end

    subgraph UNIVERSALI [Porte Universali]
        P_NAND["NAND (NOT-AND)"]
        P_NOR["NOR (NOT-OR)"]
    end

    subgraph SPECIALI [Porte Aritmetiche]
        P_XOR["XOR (OR Esclusivo)"]
        P_XNOR["XNOR (Equivalenza)"]
    end
```

1. Porta NOT (Invertitore)

Produce in uscita il complemento logico dell'unico ingresso applicato.

- **Espressione algebrica:** X = A (oppure X = A′)

Ingresso A	Uscita X = A
0	1
1	0

---

2. Porta AND (Prodotto Logico)

L'uscita è 1 **solo se tutti gli ingressi sono contemporaneamente 1**. Se anche un solo ingresso è 0, l'uscita è 0.

- **Espressione algebrica:** $X = A \cdot B$ (oppure $X = AB$)

A	B	Uscita $X = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	**1**

---

3. Porta OR (Somma Logica)

L'uscita è 1 **se almeno uno degli ingressi è 1**. L'uscita è 0 unicamente se tutti gli ingressi sono a 0.

- **Espressione algebrica:** $X = A + B$

A	B	Uscita $X = A + B$
0	0	**0**
0	1	1
1	0	1
1	1	1

---

4. Porta NAND (Negazione di AND)

È una porta AND seguita da un invertitore. L'uscita è 0 solo quando tutti gli ingressi sono 1. In tutti gli altri casi l'uscita è 1.

- **Espressione algebrica:** $X = A \cdot B$

A	B	Uscita X = AB
0	0	1
0	1	1
1	0	1
1	1	**0**

> **Proprietà Universale della porta NAND**
>
> La porta NAND è detta **porta universale** perché combinando solo porte NAND è possibile realizzare qualsiasi altra funzione logica (NOT, AND, OR, XOR, ecc.).

---

5. Porta NOR (Negazione di OR)

È una porta OR seguita da un invertitore. L'uscita è 1 esclusivamente se tutti gli ingressi sono contemporaneamente a 0.

- **Espressione algebrica:** X = A + B

A	B	Uscita X = A + B
0	0	**1**
0	1	0
1	0	0
1	1	0

---

6. Porta XOR (Exclusive OR - Disgiunzione Esclusiva)

L’uscita è 1 **se e solo se gli ingressi sono diversi tra loro** (uno a 0 e l’altro a 1). Se gli ingressi sono identici, l’uscita vale 0.

- **Espressione algebrica:** $X = A \oplus B = AB + \overline{A}\,\overline{B}$

A	B	Uscita $X = A \oplus B$
0	0	0
0	1	**1**
1	0	**1**
1	1	0

---

7. Porta XNOR (Coincidenza o Equivalenza)

L’uscita è 1 **se gli ingressi sono identici/uguali** (`00` oppure `11`). Se gli ingressi sono discordi, l’uscita vale 0.

- **Espressione algebrica:** $X = A \oplus B = AB + \overline{A}\,\overline{B}$

A	B	Uscita $X = A \oplus B$
0	0	**1**
0	1	0
1	0	0
1	1	**1**

---

3. Analisi delle Reti Combinatorie

Esercizio 1: Dallo Schema alla Funzione Booleana

Consideriamo il circuito studiato negli appunti:

1. Gli ingressi A e B entrano in una porta AND $\Longrightarrow$ $Y = A \cdot B$.
2. L'uscita intermedia Y e il terzo ingresso C entrano in una porta OR $\Longrightarrow$ $Z = Y + C = (A \cdot B) + C$.
3. L'uscita Z passa attraverso una porta NOT $\Longrightarrow$ $X = Z = (A \cdot B) + C$.

```mermaid
flowchart LR
    A["A"] --> AND1["AND"]
    B["B"] --> AND1
    AND1 -->|Y = A·B| OR1["OR"]
    C["C"] --> OR1
    OR1 -->|Z = (A·B) + C| NOT1["NOT"]
    NOT1 --> OUT["X = ¬((A·B) + C)"]
```

Tavola di Verità Completa del Circuito:

A	B	C	$Y = A \cdot B$	$Z = Y + C$	$X = Z$
0	0	0	0	0	**1**
0	0	1	0	1	**0**
0	1	0	0	0	**1**
0	1	1	0	1	**0**
1	0	0	0	0	**1**
1	0	1	0	1	**0**
1	1	0	1	1	**0**
1	1	1	1	1	**0**

Esercizio 2: Dalla Funzione Booleana allo Schema Circuitale

Consideriamo l'espressione logica: $F = A + B + C$

Procedura di Sintesi a Blocchi:

1. Somma tra B e C tramite porta OR $\Longrightarrow$ $(B + C)$.
2. Negazione del risultato tramite porta NOT $\Longrightarrow$ $B + C$ (oppure con porta NOR diretta).
3. Negazione dell'ingresso A tramite porta NOT $\Longrightarrow$ A.
4. Somma tra A e $B + C$ tramite porta OR $\Longrightarrow$ $A + B + C$.
5. Negazione finale dell'intera espressione tramite porta NOT $\Longrightarrow$ $F = A + B + C$.

```
flowchart LR
    B["B"] --> OR_BC["OR"]
    C["C"] --> OR_BC
    OR_BC --> NOT_BC["NOT"]

    A["A"] --> NOT_A["NOT"]

    NOT_A -->|¬A| OR_MAIN["OR"]
    NOT_BC -->|¬(B+C)| OR_MAIN

    OR_MAIN -->|¬A + ¬(B+C)| NOT_FINAL["NOT"]
    NOT_FINAL --> F["F = ¬(¬A + ¬(B+C))"]
```

Semplificazione Algebrica con i Teoremi di De Morgan:

Applicando il teorema di De Morgan $X + Y = X \cdot Y$: $F = A \cdot B + C = A \cdot (B + C) = A \cdot B + A \cdot C$

> **Vantaggio della Semplificazione**
>
> Con la forma semplificata $F = A \cdot (B + C)$ servono solamente **due porte logiche** (un OR e un AND), risparmiando componenti, spazio su circuito stampato, consumo energetico e tempo di propagazione!

---

4. Teoremi Fondamentali dell'Algebra di Boole

Nome del Teorema	Forma AND (Prodotto)	Forma OR (Somma)
Identità	$A \cdot 1 = A$	$A + 0 = A$
Elemento Nullo	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotenza	$A \cdot A = A$	$A + A = A$
Complementarietà	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Involuzione (Doppia Negazione)	$\overline{\overline{A}} = A$	-
Commutatività	$A \cdot B = B \cdot A$	$A + B = B + A$
Associatività	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
Distributività	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Teoremi di De Morgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$

Proprietà e Classificazione dei Sistemi

> **Perché si classificano i sistemi?**
>
> Le proprietà dei sistemi servono a categorizzare il loro comportamento matematico e operativo. Classificare un sistema consente di scegliere il modello formale corretto (es. equazioni differenziali, automi a stati finiti, circuiti logici combinatori o sequenziali) per analizzarlo, simularlo o progettarlo.

```mermaid
flowchart TD
    S["Classificazione di un Sistema"]
    S --> T["Tempo ed Evoluzione"]
    S --> M["Memoria e Dipendenza"]
    S --> N["Origine e Natura"]
    S --> C["Certezza dei Risultati"]
    S --> V["Tipo di Valori Trattati"]

    T --> P1["Variante vs Invariante"]
    M --> P2["Statico (Combinatorio) vs Dinamico (Sequenziale)"]
    M --> P3["Proprio (Moore) vs Improprio (Mealy)"]
    N --> P4["Naturale vs Artificiale"]
    C --> P5["Deterministico vs Stocastico"]
    V --> P6["Continuo vs Discreto"]
```

---

1. Statico vs Dinamico (Memoria del Sistema)

Questa proprietà indica se il sistema ha o meno una **memoria** del proprio passato.

Sistema Statico (Senza Memoria)

- **Definizione:** L'uscita a un qualsiasi istante t dipende **esclusivamente** dal valore dell'ingresso applicato nello stesso istante t: $U(t) = g(I(t))$

- **Caratteristiche:** Il sistema non possiede stati interni; cessato lo stimolo in ingresso, l'effetto svanisce all'istante.
- **Esempio:**
  - Una resistenza elettrica ($V = R \cdot I$, la tensione ai capi dipende all'istante dalla corrente).
  - Una porta logica elementare (es. porta AND o invertitore NOT).

Sistema Dinamico (Con Memoria)

- **Definizione:** L'uscita a un istante t dipende non solo dall'ingresso attuale $I(t)$, ma anche dalla **storia passata** degli ingressi, sintetizzata nello **stato interno** $S(t)$:
  $U(t) = g(S(t), I(t))$ oppure $U(t) = g(S(t))$
- **Caratteristiche:** Il sistema memorizza eventi trascorsi.
- **Esempio:**
  - Un contatore di accessi (il conteggio attuale dipende da quante persone sono passate prima).
  - Un condensatore elettrico (immagazzina carica nel tempo).
  - Un computer o un flip-flop.

---

2. Combinatorio vs Sequenziale

È la formulazione adottata in **elettronica digitale e informatica** per i sistemi statici e dinamici:

```
flowchart LR
    subgraph Combinatorio [Sistema Combinatorio]
        direction LR
        In1["Ingresso I(t)"] --> ReteComb["Rete Logica Combinatoria"] --> Out1["Usci
    end
```

```mermaid
flowchart LR
    subgraph Sequenziale [Sistema Sequenziale]
        direction LR
        In2["Ingresso I(t)"] --> ReteSeq["Logica / ALU"] --> Out2["Uscita U(t)"]
        ReteSeq --> Reg[("Memoria / Registri: S(t)")]
        Reg -->|Feedback Stato Corrente| ReteSeq
    end
```

- **Sistema Combinatorio:** Le uscite sono una funzione logica pura degli ingressi correnti. Se gli ingressi tornano allo stato iniziale, le uscite ritornano istantaneamente ai valori corrispondenti. Non c'è feedback né elementi di memorizzazione.
- **Sistema Sequenziale:** Contiene celle di memoria (flip-flop, latch, registri). L'uscita futura dipende dalla sequenza temporale di tutti gli stimoli ricevuti in passato.

---

3. Naturale vs Artificiale

Classificazione basata sull'origine fisica del sistema:

- **Sistema Naturale:** Esiste e opera in natura spontaneamente, senza l'intervento costruttivo dell'uomo.
  - *Esempi:* Il cuore umano, il sistema cardiocircolatorio, il sistema solare, un ecosistema fluviale.
- **Sistema Artificiale:** Progettato, assemblato e realizzato dall'essere umano per eseguire funzioni specifiche.
  - *Esempi:* Un computer, un'automobile, un distributore automatico di bevande, un protocollo di rete.

---

4. Deterministico vs Stocastico (Probabilistico)

Classificazione basata sulla prevedibilità matematica del comportamento:

Sistema Deterministico

- **Definizione:** A partire da uno stato iniziale noto S(t0) e applicando la medesima sequenza di ingressi, il sistema produrrà **sempre e con certezza assoluta** la stessa evoluzione degli stati

e le stesse uscite.
- **Esempio:** Un microprocessore, un algoritmo deterministico, un circuito logico digitale.

Sistema Stocastico (Probabilistico)

- **Definizione:** Il comportamento del sistema è soggetto a variabili casuali, rumore o distribuzioni di probabilità. A parità di condizioni iniziali e ingressi, l'uscita non è determinabile a priori in modo univoco, ma solo in termini statistici.
- **Esempio:**
  - Il lancio di un dado o la roulette.
  - Il modello di arrivo dei pacchetti su un router Internet (traffico poissoniano).
  - Le previsioni meteorologiche.

---

5. Continuo vs Discreto

Riguarda il dominio del **tempo** e l'insieme dei **valori** assunti dalle grandezze:

```mermaid
flowchart TD
    CD["Classificazione Continuo / Discreto"]
    CD --> Tempo["Rispetto al Tempo"]
    CD --> Grandezza["Rispetto alle Variabili (Ampiezza)"]

    Tempo --> TC["Tempo Continuo (t ∈ ℝ)"]
    Tempo --> TD["Tempo Discreto (t ∈ ℤ o passi k)"]

    Grandezza --> GC["A Valori Continui (Insieme infinito non numerabile)"]
    Grandezza --> GD["A Valori Discreti (Insieme finito o numerabile: es. 0/1)"]
```

Tipologia	Caratteristiche	Esempio Pratico
Continuo	Le grandezze variano in modo continuo nel tempo e possono assumere qualsiasi valore all'interno di un intervallo di numeri reali (R).	Segnale vocale captato da un microfono a membrana, temperatura atmosferica registrata da termometro a mercurio.
Discreto	Le variabili assumono solo valori appartenenti a un insieme finito o numerabile, e/o il tempo scorre a intervalli discreti (scandito da un clock).	Orologio digitale, file binario, tastiera, registri della CPU.

---

6. Variante vs Invariante nel Tempo (Stazionario)

Indica se le regole intrinseche e i parametri fisici del sistema cambiano col passare del tempo.

- **Sistema Invariante (Stazionario):** Il comportamento non dipende dall'istante temporale in cui viene applicato lo stimolo. Se applichiamo una data sequenza di ingressi oggi o fra una settimana, il sistema risponderà nello stesso identico modo (a parità di stato iniziale), subendo unicamente una traslazione temporale: $I(t - \tau) \implies U(t - \tau)$
  - *Esempio:* Un circuito logico ideale a temperatura controllata.
- **Sistema Variante nel Tempo:** I parametri interni cambiano nel tempo a causa di usura, degradazione, invecchiamento o variazioni dell'ambiente esterno.
  - *Esempio:* I componenti elettronici soggetti a surriscaldamento o esaurimento (una batteria la cui resistenza interna sale con i cicli di carica).

---

7. Proprio (Moore) vs Improprio (Mealy)

Questa distinzione riguarda la dipendenza diretta dell'uscita negli automi e nei sistemi dinamici:

```
classDiagram
    class Sistema_Proprio_Moore {
        +U(t) = g(S(t))
        Uscita legata SOLO allo stato interno
        Transizioni stabili sincronizzate al clock
    }
    class Sistema_Improprio_Mealy {
        +U(t) = g(S(t), I(t))
        Uscita legata a STATO e INGRESSO istantaneo
        Risposta immediata ma sensibile ai disturbi
    }
```

- **Sistema Proprio (Modello di Moore):** L'uscita dipende **esclusivamente dallo stato interno** presente. L'ingresso influenza lo stato futuro, ma non altera direttamente l'uscita corrente nello stesso istante. $U(t) = g(S(t))$
- **Sistema Improprio (Modello di Mealy):** L'uscita dipende **contemporaneamente sia dallo stato interno corrente sia dall'ingresso attuale**. Una variazione dell'ingresso può riflettersi immediatamente sull'uscita senza attendere il ciclo successivo. $U(t) = g(S(t), I(t))$

---

8. Tavola Sinottica di Classificazione Rapida

Criterio	Valore A	Valore B	Domanda Guida per Riconoscerlo
Memoria	Statico	Dinamico	*L'uscita dipende solo dall'adesso o ricorda gli ingressi passati?*
Origine	Naturale	Artificiale	*È stato creato dall'uomo o esiste già nell'universo?*
Certezza	Deterministico	Stocastico	*Lo stesso identico input dà SEMPRE lo stesso identico output?*
Grandezze	Continuo	Discreto	*I valori possono assumere qualsiasi frazione reale o sono passi/livelli finiti?*
Stazionarietà	Invariante	Variante	*Il comportamento del sistema si degrada o cambia nel tempo?*
Logica	Combinatorio	Sequenziale	*Il circuito contiene elementi di memoria (registri, flip-flop)?*
Uscita	Proprio (Moore)	Improprio (Mealy)	*L'uscita cambia istantaneamente se muovo l'ingresso senza cambiare stato?*